SCHEDULING ALGORITHMS :

1. fcfs :

```
maitreyee@localhost:~
[maitreyee@localhost ~]$ ls
-        bank       Desktop    mait     Music      Pictures    sh          Videos
a        banker.c   Documents  maithu   os_lab     Public      sjf         while.sh
ABC      bankers    Downloads  maitre   os_lab1    REC         sjf.c       whilw.sh
abc.sh   bm         h          mash     os_lab1.c  sample.c    Templates   XYZ
ab.sh    close      mai        mithii   os_lab.c   sample.txt  test.sh
[maitreyee@localhost ~]$ vi os_lab1.c
[maitreyee@localhost ~]$ gcc os_lab1.c -o os_lab1
[maitreyee@localhost ~]$ ./os_lab1
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8

Average Turnaround Time: 8.67
Average Waiting Time: 3.33
[maitreyee@localhost ~]$
```

```
Activities   Terminal                                   Mar 18  19:12
maitreyee@localhost:~ — /usr/bin/vi
```

```c
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];

    // Input
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    float total_tat = 0, total_wt = 0;

    // First process
    ct[0] = at[0] + bt[0];

    // Remaining processes
    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];   // CPU idle
        else
            ct[i] = ct[i-1] + bt[i];
    }

    // Calculate TAT and WT
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        total_tat += tat[i];
        total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f", total_tat / n);
    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);

    return 0;
}
```

2. sjf :

```
maitreyee@localhost:~
[maitreyee@localhost ~]$ ls
-        bank       Desktop    mait     Music      Pictures    sh          whilw.sh
a        banker.c   Documents  maithu   os_lab     Public      Templates   XYZ
ABC      bankers    Downloads  maitre   os_lab1    REC         test.sh
abc.sh   bm         h          mash     os_lab1.c  sample.c    Videos
ab.sh    close      mai        mithii   os_lab.c   sample.txt  while.sh
[maitreyee@localhost ~]$ vi sjf
[maitreyee@localhost ~]$ vi sjf.c
[maitreyee@localhost ~]$ vi sjf.c
[maitreyee@localhost ~]$ vi sjf.c
[maitreyee@localhost ~]$ gcc sjf.c -o sjf
[maitreyee@localhost ~]$ ./sjf
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 7
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 4 1

Average Turnaround Time: 7.00
Average Waiting Time: 3.00
[maitreyee@localhost ~]$
```

Code :

```
Activities   Terminal                    Mar 18  19:10
maitreyee@localhost:~ — /usr/bin/vim sjf.c
#include <stdio.h>
int main()
{
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], tat[n], wt[n];
    int done[n];

    // Input
    for(int i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    int completed = 0;
    int completionTime = 0;
    float avgtat = 0, avgwt = 0;

    // SJF Scheduling (Non-Preemptive)
    while(completed < n)
    {
        int idx = -1;
        int minBT = 9999;

        for(int i = 0; i < n; i++)
        {
            if(at[i] <= completionTime && done[i] == 0)
            {
                if(bt[i] < minBT)
                {
                    minBT = bt[i];
                    idx = i;
                }
            }
        }

        if(idx != -1)
                                                                    1,1
```

Activities Terminal Mar 18 19:10

maitreyee@localhost:~ — /usr/bin/vim sjf.c

```
        if(at[i] <= completionTime && done[i] == 0)
        {
            if(bt[i] < minBT)
            {
                minBT = bt[i];
                idx = i;
            }
        }
    }

    if(idx != -1)
    {
        if(completionTime < at[idx])
            completionTime = at[idx];

        completionTime += bt[idx];

        tat[idx] = completionTime - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        done[idx] = 1;
        completed++;
    }
    else
    {
        completionTime++;
    }
}

for(int i = 0; i < n; i++)
{
    avgtat += tat[i];
    avgwt += wt[i];
}

printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
printf("Average Waiting Time: %.2f\n", avgwt / n);

return 0;
}
```

71,0-1 Bot

3. Round robin

```
Activities   Terminal                    Mar 18  19:07
                          maitreyee@localhost:~
[maitreyee@localhost ~]$ ls
-        bank       Desktop    mait     Music      Pictures    sh         Videos
a        banker.c   Documents  maithu   os_lab     Public      sjf        while.sh
ABC      bankers    Downloads  maitre   os_lab1    REC         sjf.c      whilw.sh
abc.sh   bm         h          mash     os_lab1.c  sample.c    Templates  XYZ
ab.sh    close      mai        mithii   os_lab.c   sample.txt  test.sh
[maitreyee@localhost ~]$ vi roundrobin.c
[maitreyee@localhost ~]$ gcc roundrobin.c -o roundrobin
[maitreyee@localhost ~]$ ./round robin
bash: ./round: No such file or directory
[maitreyee@localhost ~]$ ./roundrobin
Enter number of processes:
3
Enter Time Quantum:
2
Enter AT and BT for P1:
0 5
Enter AT and BT for P2:
1 3
Enter AT and BT for P3:
2 1

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
[maitreyee@localhost ~]$
```

Code :

Activities Terminal Mar 18 19:08

maitreyee@localhost:~ — /usr/bin/vim roundrobin.c

```c
#include <stdio.h>
int main()
{
    int n, tq, completed = 0, completionTime = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);

    int at[n], bt[n], rem[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;

    // Input
    for (int i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: \n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];   // Remaining time = Burst time
    }

    // Round Robin Logic
    while (completed < n)
    {
        int idle = 1;

        for (int i = 0; i < n; i++)
        {
            if (at[i] <= completionTime && rem[i] > 0)
            {
                idle = 0;

                if (rem[i] > tq)
                {
                    completionTime += tq;
                    rem[i] -= tq;
                }
                else
                {
                    completionTime += rem[i];
```

1,1 Top

Activities Terminal Mar 18 19:08

maitreyee@localhost:~ — /usr/bin/vim roundrobin.c

```c
        int idle = 1;

        for (int i = 0; i < n; i++)
        {
            if (at[i] <= completionTime && rem[i] > 0)
            {
                idle = 0;

                if (rem[i] > tq)
                {
                    completionTime += tq;
                    rem[i] -= tq;
                }
                else
                {
                    completionTime += rem[i];
                    tat[i] = completionTime - at[i];
                    wt[i] = tat[i] - bt[i];
                    rem[i] = 0;
                    completed++;
                }
            }
        }

        if (idle)
            completionTime++;
    }

    // Calculate averages
    for (int i = 0; i < n; i++)
    {
        avgtat += tat[i];
        avgwt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
    printf("Average Waiting Time: %.2f\n", avgwt / n);

    return 0;
}
```

4. priority scheduling

```
Activities   Terminal                    Mar 18  19:18
                          maitreyee@localhost:~
[maitreyee@localhost ~]$ ls
-        banker.c   Downloads  mash       os_lab.c      sample.c    test.sh
a        bankers    h          mithii     Pictures      sample.txt  Videos
ABC      bm         mai        Music      Public        sh          while.sh
abc.sh   close      mait       os_lab     REC           sjf         whilw.sh
ab.sh    Desktop    maithu     os_lab1    roundrobin    sjf.c       XYZ
bank     Documents  maitre      os_lab1.c  roundrobin.c  Templates
[maitreyee@localhost ~]$ vi priority
[maitreyee@localhost ~]$ vi priority.c
[maitreyee@localhost ~]$ gcc priority.c -o priority
[maitreyee@localhost ~]$ ./priority
Enter number of processes:
3
Enter AT, BT and Priority for P1:
0 5 2
Enter AT, BT and Priority for P2:
1 3 1
Enter AT, BT and Priority for P3:
2 8 3

Average Turnaround Time: 8.67
Average Waiting Time: 3.33
[maitreyee@localhost ~]$
```

```
Activities   Terminal                    Mar 18  19:19
                          maitreyee@localhost:~ — /usr/bin/vim priority.c
#include <stdio.h>

int main()
{
    int n, completed = 0, time = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], priority[n], done[n];
    int ct[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;

    // Input
    for (int i = 0; i < n; i++)
    {
        printf("Enter AT, BT and Priority for P%d: \n", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &priority[i]);
        done[i] = 0;
    }

    // Scheduling Logic (Non-Preemptive Priority: lower number = higher priority)
    while (completed < n)
    {
        int idx = -1;
        int highPriority = 9999;

        for (int i = 0; i < n; i++)
        {
            if (at[i] <= time && done[i] == 0)
            {
                if (priority[i] < highPriority)
                {
                    highPriority = priority[i];
                    idx = i;
                }
            }
        }

        if (idx != -1)
        {
```

Activities Terminal Mar 18 19:19

maitreyee@localhost:~ — /usr/bin/vim priority.c

```
        if (idx != -1)
        {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            done[idx] = 1;
            completed++;
        }
        else
        {
            time++;   // CPU Idle
        }
    }

    // Average calculation
    for (int i = 0; i < n; i++)
    {
        avgtat += tat[i];
        avgwt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
    printf("Average Waiting Time: %.2f\n", avgwt / n);

    return 0;
}
```